

Pràctica de Cerca Local

Intel·ligència Artificial

2025/2026 1Q

Facultat d'Informàtica de Barcelona

Iu Franquesa Tarté

Junzhe Xia

Yaoyao Guo Zhang

Índex

1. El problema.....	3
2. Estat.....	4
3. Operadors.....	5
3.1. Afegir.....	5
3.2. Swap (entre una petició atesa i una que no).....	5
3.3. Swap (entre peticions ateses).....	5
3.4. Reassignar.....	6
4. Solució inicial.....	7
Solució buida.....	7
Solució inicial aleatòria.....	7
Solució inicial greedy.....	8
Solució inicial greedy versió 2.....	8
Comparació i recomanacions.....	9
5. Heurística.....	10
6. Optimitzacions.....	11
7. Experiments.....	12
Experiment 1 - Elecció d'operadors i heurística.....	12
Experiment 2 - Estrategia de generació de la solució inicial.....	12
Experiment 3 - Anàlisi dels paràmetres de SA.....	13
Experiment 4 - Evolució en temps.....	13
Experiment 5 - Anàlisi del comportament davant d'una reducció del nombre de centres.....	14
Experiment 6 - Anàlisi del comportament davant d'un augment en el cost per quilòmetre.....	14
Experiment 7 - Anàlisi del comportament davant d'una reducció de quilòmetres diaris.....	15
8. Conclusions.....	16
Annex.....	17
[Annex 1] - Codi font de la pràctica (GitHub).....	17
[Annex 2] - Resultats dels experiments (Google Sheet).....	17
[Annex 3] - 8.2. Competencia de trabajo en equipo: Trabajo de innovación.....	17

1. El problema

La pràctica ens planteja un problema “real”: la planificació diària de rutes de distribució.

Concretament, distribució de combustible des de diversos **centres de distribució**, mitjançant **camions** cisterna que han d’abastir un conjunt de **gasolineres** dins d’una àrea de $100 \times 100 \text{ km}^2$.

També tenim algunes restriccions. Cada gasolinera té un **nombre limitat de peticions**, i cada camió pot fer com a **màxim 5 viatges** i fins a **640 km** al dia. En cada viatge, el camió no pot omplir més de **dos dipòsits**, és a dir que només pot atendre fins a 2 peticions per viatge, ja sigui en gasolineres diferents o en la mateixa.

L’objectiu és **maximitzar el benefici** total de la companyia, considerant que el valor de cada petició disminueix amb els **dies d’espera** segons la fórmula: $\% \text{preu} = (100 - 2^{\text{dies}})\%$, però en cas que la petició s’atengui al mateix dia, el preu serà lleugerament més alt: $\% \text{preu} = 102\%$. També, cal minimitzar la **distància total recorreguda**, ja que cada quilòmetre té un cost fix de 2 unitats monetàries.

L’espai de cerca està format per totes les possibles assignacions de peticions als camions i totes les combinacions de rutes que compleixin les restriccions de distància i viatges. Aquest és un problema de cerca local perquè:

- L’espai de solucions és molt ampli i no estructurat, fet que impossibilita una cerca exhaustiva.
- No existeix un estat final conegut, ja que “sempre” es pot millorar una solució.
- Els canvis entre estats són locals i incrementals, propis de tècniques de millora iterativa.
- La funció heurística permet comparar directament la qualitat de dues solucions.

En resum, es tracta d’un problema d’optimització que busca equilibrar beneficis i costos sota limitacions espacials i temporals. A causa de la seva complexitat i del gran nombre de possibles configuracions, és especialment adequat per ser resolt mitjançant tècniques de cerca local, que permeten trobar solucions bones i eficients sense necessitat d’obtenir l’òptim global.

Per resoldre el problema, podem usar algoritmes com Hill Climbing o Simulated Annealing, ideals per aquesta cerca local.

2. Estat

Per a la representació d'estats del problema hem escollit els següents elements que en formaran part:

- Un `ArrayList<Distribucion>` `camions` que representa les coordenades dels centres de distribució dels diferents camions. Si hi han múltiples camions en un mateix centre, les seves coordenades estaran repetides. Hem escollit un `ArrayList` ja que es la forma més senzilla i eficaç per a obtenir una representació de un conjunt d'elements.
- Un `ArrayList<Gasolinera>` `gasolineras` que representa les coordenades de les gasolineres (cada una amb el seu conjunt de peticions). També hem escollit un `ArrayList` pel mateix motiu que l'element anterior.
- Un `ArrayList<boolean[]>` `gasolineras_info` que representa si una petició de una gasolinera ha estat atesa o no. Hem escollit un `ArrayList<boolean[]>` ja que és la forma més senzilla i eficaç per a obtenir una representació que ens indica per cada gasolinera les seves peticions (si estan ateses o no). No es podria fer amb una matriu fixe ja que cada gasolinera pot tenir un número diferent de peticions.
- Una classe `Viaje` que representa un viatge de un camió. Aquest viatge pot anar a parar a com a molt dos gasolineres diferents o a dues iguals, però mai més de 2. Dintre d'aquesta classe tenim informació de quines gasolineres s'han visitat i l'índex de les peticions ateses d'aquelles gasolineres. També tenim el número de km recorreguts del viatge.
- Dos matrius auxiliars amb les distàncies precomputades en construir el board: `int[][] distCentroGas` (distància de cada centre a cada gasolinera) i `int[][] distGasGas` (distància de cada gasolinera a cada gasolinera) perquè s'han de consultar molt sovint. Hem escollit matrius ja que hi ha una mida fixa de gasolineres i centres de distribució.
- Un `ArrayList<ArrayList<Viaje>>` `viajesPorCamion` que representa els viatges que ha fet cada camió. Hem escollit `ArrayList<ArrayList<>>` ja que és la forma més senzilla i eficaç per a obtenir una representació que ens indica per cada camió els viatges que té. No es podria fer amb una matriu ja que cada camió pot tenir un número diferent de viatges. Encara que cada camió pot fer fins a 5 viatges, hem decidit que un viatge només existeix si el camió efectivament atén alguna petició en aquell viatge.
- Un `int[] kmsPorCamion` que representa els quilòmetres que ha recorregut cada camió.
- El benefici actual (tenint en compte la penalització de les peticions no ateses al final del dia), que s'utilitzarà per calcular l'heurística.
- El benefici d'avui (sense tenir en compte la penalització de les peticions no ateses), que és el valor que s'utilitza en els experiments.
- La suma total dels quilòmetres recorreguts de tots els camions.

Per mesurar una aproximació de l'espai de búsqueda, necessitem trobar el nombre de possibles combinacions d'assignacions de peticions (P) als viatges dels camions. Seguint les restriccions de l'enunciat, direm que en total, es poden atendre com a màxim $N = 10C$ peticions.

Aquest número es pot representar amb la notació combinatòria com:
$$\binom{P}{N} = \frac{P!}{N!(P - N)!}$$

Tot i així, ja que sabem que $P \gg N$ en la majoria dels casos, podem simplificar aquest càlcul, aproximant-lo a:

$$\binom{P}{N} \approx \frac{P^N}{N!}$$

3. Operadors

Per aquest apartat, on explicarem els diferents operadors que hem implementat, definim algunes nomenclatures útils: $C = \# \text{camions}$ i $G = \# \text{gasolineres}$. Com que el nombre de peticions i el nombre de viatges són proporcionals a G i C respectivament, no les utilitzarem en l'anàlisi de costos i factors de ramificació. Sabem també que $G \gg C$ en la majoria de casos.

3.1. Afegir

Funció: Afegir una petició a un viatge.

Precondicions: La petició no ha estat atesa encara, el viatge existeix i té menys de 2 parades i no s'incompleixen les restriccions del problema.

Condicions d'aplicabilitat: Es pot aplicar sempre i quan es compleixin les precondicions.

Factor de ramificació: $O(C * G)$

Hem escollit aquest operador per poder gestionar la [Solució buida](#), de la que parlarem més endavant, i també per complementar un possible operador capaç d'eliminar una petició d'un viatge (finalment no implementat).

Aquest operador és el que té més velocitat de convergència amb diferència d'entre els operadors escollits.

3.2. Swap (entre una petició atesa i una que no)

Funció: Intercanviar 2 peticions, una d'atesa i una altre que no.

Precondicions: Una petició ha estat atesa i l'altre no, el viatge existeix i no s'incompleixen les restriccions del problema.

Condicions d'aplicabilitat: Es pot aplicar sempre i quan es compleixin les precondicions.

Factor de ramificació: $O(C * G)$

Hem escollit aquest operador per poder optimitzar una solució ja decent. Juntament amb el seu anàlog per peticions ateses, s'usa especialment per anar escalant lentament la qualitat de la solució, sense grans salts.

3.3. Swap (entre peticions ateses)

Funció: Intercanviar 2 peticions ateses. És a dir, camió-1 passa a atendre petició-2 i viceversa.

Precondicions: Les peticions han estat ateses i no són la mateixa, els viatges existeixen i no s'incompleixen les restriccions del problema.

Condicions d'aplicabilitat: Es pot aplicar sempre i quan es compleixin les precondicions.

Factor de ramificació: $O(C^2)$

Hem escollit aquest operador per poder optimitzar una solució ja decent. Juntament amb el seu anàlog per peticions no ateses, s'usa especialment per anar escalant lentament la qualitat de la solució, sense grans salts.

3.4. Reassignar

Funció: Reassignar una petició atesa d'un viatge d'un camió a un viatge (potencialment nou) d'un altre camió.

Precondicions: La petició ha estat atesa prèviament en un viatge del camió origen, el camió destí existeix i no s'incompleixen les restriccions del problema en afegir la petició al camió destí.

Condicions d'aplicabilitat: Es pot aplicar sempre i quan es compleixin les precondicions.

Factor de ramificació: $O(C^2)$

Aquest operador va ser inicialment implementat per permetre redistribuir peticions entre camions quan la solució inicial no era òptima en termes de distribució geogràfica. No obstant, va ser descartat perquè vam observar que rarament s'utilitzava durant la cerca. El motiu principal és que les solucions inicials greedy i random ja assignaven les peticions de forma relativament eficient, llavors la probabilitat que reassignar una petició a un altre camió millori la solució és molt baixa. Els operadors swap resulten més efectius per optimitzar l'assignació, ja que permeten intercanvis locals sense alterar dràsticament la distribució geogràfica inicial.

4. Solució inicial

Hem implementat **4 estratègies** diferents per calcular les solucions inicials: solució **buida**, solució **aleatòria** i dues variants de **greedy**, una prioritzant les peticions amb menys dies pendents i una altra d'igual, però també minimitzant la distància recorreguda. Totes les solucions inicials son solucions vàlides.

A priori, amb la teoria a la mà, és raonable pensar que les solucions greedy donaran millors resultats amb Hill Climbing ja que són suficientment bones per convergir ràpidament en un mínim local. Per altra banda, la solució buida i l'aleatòria haurien de donar bons resultats amb Simulated Annealing per sobre de Hill Climbing.

Solució buida

L'algorisme per obtenir una solució inicial buida genera una solució vàlida que compleix les restriccions del problema. És un punt de partida completament neutre, permetent que la construcció de la solució estigui totalment guiada per la funció heurística de la cerca. L'algorisme de solució buida simplement inicialitza l'estat amb: (des de la constructora de board)

- Camions sense viatges assignats. (Cap petició atesa)
- Benefici inicial = 0.

El **cost** de construir-la és molt baix, $O(1)$, només inicialitzant estructures buides pero el trade-off implica requerir moltes més iteracions de cerca per arribar a solucions decents, ja que la **qualitat** inicial és molt baixa (la pitjor).

Solució inicial aleatòria

L'algorisme per obtenir una solució inicial aleatòria genera una solució vàlida que compleix les restriccions del problema. L'ordre que segueix per a la generació és el següent:

- Crea una llista amb totes les peticions pendents i les barreja aleatòriament.
- Per cada petició, escull un camió inicial aleatori i intenta assignar-la si no trenca les restriccions del problema.
 - Si el camió està buit, crea un viatge nou sempre i quan els kilòmetres d'anada i tornada no superin el límit de 640 km diaris.
 - Si el camió ja té viatges i l'últim viatge només té una parada, intenta afegir la petició com a segona parada del viatge respectant el límit de km.
 - Si l'últim viatge ja té dues parades, intenta crear un viatge nou sempre que el camió tingui menys de 5 viatges.

El **cost** d'aquesta estratègia és $O(P \times C)$, on P és el nombre total de peticions i C el nombre de camions. El shuffle inicial costa $O(P \log P)$, però després per cada petició només es proven fins a C camions amb operacions de cost constant. En la pràctica, amb P de l'ordre de centenars i C petit (10-30), el temps real és negligible (menys d'1 ms per instàncies típiques). En definitiva, el cost de trobar aquesta solució és baix comparat amb el temps de la cerca posterior, oferint un trade-off òptim entre rapidesa i diversitat.

Pel que fa a la **qualitat** de la solució, aquesta estratègia genera solucions de bondat mitjana-baixa ja que no optimitza cap criteri específic, però té l'avantatge d'oferir alta diversitat entre execucions, permetent a Hill Climbing explorar diferents regions de l'espai de solucions i evitar convergència prematura a mínims locals.

Solució inicial greedy

L'ordre que segueix l'algorisme greedy clàssic és el següent:

- Utilitza una cua de prioritat que ordena totes les peticions per pèrdua potencial decreixent, calculada com la diferència entre el benefici d'atendre la petició avui i el benefici d'atendre-la demà.
- L'assignació es fa camió per camió en ordre seqüencial (0, 1, 2, ...). Per cada camió, mentre quedin peticions i no s'arribi al límit de km, es treu la petició amb més pèrdua potencial de la cua i s'intenta assignar.
 - Si el camió està buit es crea un viatge nou
 - Si l'últim viatge té només una parada s'afegeix com a segona parada, i si l'últim viatge ja té dues parades es crea un nou viatge.
 - Quan no es pot assignar una petició per límit de km, es passa al següent camió.

Aquesta estratègia tendeix a omplir cada camió fins al límit abans de passar al següent, podent deixar camions buits i generant una distribució desigual de la càrrega.

El **cost** computacional és $O(P \log P)$: crear la cua de prioritat costa $O(P \log P)$, i després les P extraccions de la cua costen $O(\log P)$ cadascuna, amb operacions de comprovació i assignació de cost constant. Asimptòticament és més eficient que random per P grans, però en la pràctica la diferència és negligible per $P < 1000$.

La **qualitat** de la solució pot ser alta (potser hi ha càrrega desigual) perquè prioritza peticions urgents (amb més dies pendants), generant solucions relativament bones des del principi. Tanmateix, com ja s'ha comentat, això presenta el risc que amb Hill Climbing la solució sigui massa bona i quedi atrapada en un mínim local.

Solució inicial greedy versió 2

L'algorisme greedy millorat utilitza la mateixa cua de prioritat ordenada per pèrdua potencial que greedy 1, però aplica una estratègia d'assignació més intel·ligent. Aquesta diferència clau fa que greedy 2 optimitzi tant el temps (peticions urgents) com la distància (quilòmetres mínims), distribuint la càrrega de manera més equilibrada entre camions.:

- Per cada petició de la cua calcula quin camió té el centre de distribució més proper a la gasolinera corresponent i intenta assignar-la primer a aquest camió preferit.
 - Si el camió preferit no pot atendre la petició, prova amb la resta de camions seguint aquesta ordenació.

El **cost** computacional és $O(P \log P + P \times C)$: la cua de prioritat costa $O(P \log P)$, i per cada petició cal trobar el camió mínim amb cost $O(C)$ i potencialment provar fins a C camions. Si C és petit (10-30), el terme dominant és $O(P \log P)$, fent-lo similar en la pràctica a greedy 1. En resum, el cost de trobar aquesta solució és ràpid i l'overhead mínim de calcular el camió preferit ofereix la millor relació qualitat/cost de totes les estratègies.

La **qualitat** de la solució és molt alta, sent millor que la resta d'estratègies, però això comporta un risc elevat de quedar atrapat en mínims locals amb Hill Climbing. Per contra, aquesta estratègia és ideal per Simulated Annealing ja que SA pot escapar de mínims locals acceptant empitjoraments temporals.

Comparació i recomanacions

Comparant les quatre estratègies:

- Buida té cost $O(1)$ amb qualitat molt baixa però diversitat màxima.
- Random té cost $O(P \times C)$ amb qualitat mitjana i alta diversitat.
- Greedy 1 té cost $O(P \log P)$ amb qualitat alta però baixa diversitat.
- Greedy 2 té cost $O(P \log P)$ amb qualitat molt alta però baixa diversitat i alt risc de mínim local.

Per Hill Climbing, a priori creiem que la millor opció és la random perquè evita mínims locals gràcies a la diversitat, té cost negligible i fa múltiples execucions explorant diferents regions de l'espai de solucions. Greedy 1 és una alternativa si es vol convergència més ràpida acceptant menys exploració, però s'ha d'evitar greedy 2 per ser massa bona (queda encallada) i la buida per convergència és molt lenta.

Per Simulated Annealing, a priori creiem que la millor opció és greedy 2 perquè SA pot escapar de mínims locals, començar amb una bona solució accelera la convergència, i la temperatura inicial permet empitjoraments temporals. Random és una alternativa que ofereix major diversitat si SA té una temperatura molt alta, i la solució buida és possible només amb temperatura inicial molt alta tot i requerir moltes més iteracions.

En conclusió, la tria de l'estratègia inicial depèn críticament de l'algoritme de cerca utilitzat, el trade-off entre qualitat inicial i capacitat d'escapar mínims locals, i el temps de còmput disponible. Per aquest problema, random amb Hill Climbing i greedy 2 amb Simulated Annealing ofereixen el millor equilibri entre cost computacional i qualitat de la solució final.

5. Heurística

Per l'heurística, hem utilitzat una expressió calculada a l'estat del problema, representant el benefici total acumulat en un estat, tenint en compte tant els costos per distància com les pèrdues futures per peticions no ateses el dia actual.

Concretament, la fórmula per l'heurística d'un estat e (en unitats monetàries), es pot escriure com:

$h(e) = -\text{benefici_actual}(e)$

on $\text{benefici_actual}(e) = \Sigma(\text{ingressos per peticions ateses}) - \Sigma(\text{costos per km}) - \Sigma(\text{pèrdues per peticions no ateses avui})$.

L'heurística es negativa ja que volem maximitzar els beneficis i en aquest cas, tant Hill Climbing com Simulated Annealing busquen mínims locals.

Desglossant cada terme:

- Ingressos: per cada petició atesa, sumem el preu del dipòsit penalitzat pel temps: $\text{preuDipòsit}(t)$, on t són els dies que la petició concreta porta pendent i $\text{preuDipòsit}()$ és el preu corresponent al dipòsit per atendre la petició.
- Costos: per cada viatge, restem $d \times \text{costePorKm}$, on d és la distància recorreguda total per atendre aquella petició i $\text{costePorKm} = 2$.
- Pèrdues: per cada petició no atesa avui, restem la diferència de benefici entre atendre-la avui o demà: $\text{preuDipòsit}(t) - \text{preuDipòsit}(t+1)$.

Vam considerar també una versió simplificada que no depèn de l'estat sencer del problema, és a dir, que no depèn dels sumatoris, sinó avaluant cada operador individualment, calculant només l'impacte que genera l'operador:

$\text{benefici_local} = \text{preuDipòsit}(t) - d \times \text{costePorKm} - \text{calcPerdua}(t)$

on $\text{calcPerdua}(t) = \text{preuDipòsit}(t) - \text{preuDipòsit}(t+1)$

De totes maneres ens hem acabat decantant per la primera, ja que, tot i ser una mica més difícil de calcular, ens retorna una mesura real de la optimalitat de la solució en conjunt. Tampoc ens afegeix un gran cost, ja que tots els operadors tenen un cost desestimable.

Aquesta heurística ens permet una cerca estrictament creixent en quant al benefici, penalitzant les accions “dolentes” com la distància recorreguda i les peticions no ateses aquell dia (assumint que s'atenen al següent).

6. Optimitzacions

L'algoritme de Hill Climbing simple que vam programar a l'inici de la pràctica no era capaç de treure una solució final en temps raonable, així que vam haver d'aplicar algunes optimitzacions per solucionar aquest problema.

El cost de reduir el temps d'execució l'ha patit sobretot a la qüestió d'optimalitat de la solució, però, com sempre en algorismes iteratius sobre l'espai de solucions, es pot retocar el codi per buscar el màxim/mínim local real (i no un de fabricat per nosaltres) si així ho requerim.

Concretament, hem aplicat un total de 3 optimitzacions:

- **Retorn precipitat:** Enlloc de buscar tots els successors prometedors, i llavors escollir el millor d'entre ells, el nostre programa usa el primer que comporta una millora de la solució.
- **Limit d'iteracions:** Hem limitat per una variable el nombre d'iteracions que es poden fer sobre un mateix operador, amb tal d'evitar mirar tots els casos si resulta que és inútil, o no es pot aplicar (en general).

Aquesta és l'optimització més important, i alterant el valor d'aquesta variable podem moure'ns en el paradigma temps/optimalitat segons ens convingui.

No l'hem usat pels experiments.

També hem plantejat una millora per SA: (també útil per HC amb inicialització random)

- **Múltiples execucions:** Normalment en conjunt amb l'optimització anterior, podem executar varies vegades el programa amb canvis mínims (ja sigui per una seed concreta o per seeds random) i quedar-nos amb el millor resultat. D'aquesta manera busquem evitar localitats traicioneres i mínims/màxims poc óptims.

Encara i així, per als experiments no l'hem utilitzat per tractar de veure quins beneficis podia aconseguir donat un problema en concret.

7. Experiments

A continuació s'expliquen els experiments fets en aquesta pràctica. A tots ells hem fet 10 execucions de cada cas i hem utilitzat seeds aleatoris per a la generació dels arrays de Gasolineras i CentrosDistribucion per garantir una major aleatorietat i precisió en els resultats.

Es pot consultar l'excel amb el resultat dels experiments: [\[Annex 2\] - Resultats dels experiments \(Google Sheet\)](#)

Experiment 1 - Elecció d'operadors i heurística

L'heurística està clara, ve donada per l'enunciat, que són els beneficis obtinguts per atendre les gasolineres, restant-li els costos pels kms recorreguts i les pèrdues que suposen no atendre una petició el dia actual. No vam provar més heurístiques, ja que aquesta tenia en consideració tots els elements del problema, amb les ponderacions extremes directament de l'enunciat.

Els operadors finals són els que s'expliquen a la pràctica: el d'afegir petició i els dos d'intercanvi. Inicialment també estava el de reassignar, però ens vam adonar que quasi mai s'utilitzava, llavors el vam descartar. Però abans també n'havíem definit un que al final va resultar innecessari, el de swap de dos gasolineres d'un mateix viatge, perquè a cada viatge el camió només pot anar fins a 2 gasolineres diferents, llavors canviar-ne l'ordre no afecta la distància recorreguda.

En un inici teníem un conjunt d'operadors format per els operadors actuals i 3 més:

- Eliminar una petició d'un viatge.
- Reassignar: Moure una petició atesa a d'un viatge a un altre que no estigués complert.
- Swap (entre les 2 peticions d'un mateix viatge): Intercanviar l'ordre en que es visiten les 2 peticions ateses en un mateix viatge.

Al realitzar experiments, els resultats van mostrar les següents conclusions:

- L'operador eliminar sempre empitjora l'heurística ja que provoca que una petició atesa per un camió passés a estar no atesa. L'única forma de que l'heurística millorés és que el benefici tret de la petició fos negatiu (quasi impossible). Per tant, no s'utilitzava mai en l'execució, ergo, el no valia la pena mantindre's.
Potser en SA hagués pogut tenir alguna funció, però no el vam acabar considerant.
- L'operador de reassignar té un problema similar al d'eliminar, i és que no s'usava mai a la pràctica. Si algun camió tenia un espai buit, per heurística, és millor emplenar-lo amb una petició no atesa que amb una que sí que ho està.
- L'operador de swap (entre les 2 peticions d'un mateix viatge) tenia un problema diferent: per ignorància al proposar-lo, no vam caure en el fet que literalment no alterava el resultat del problema en una sola unitat monetaria: es segueix recorrent els mateixos quilòmetres i atenent les mateixes peticions.

Experiment 2 - Estrategia de generació de la solució inicial

En aquest experiment realitzem 10 execucions per a cada inicialització utilitzant l'algorisme de Hill Climbing i una seed aleatòria, amb 100 gasolineres, 10 centres de distribució i multiplicitat 1. Els resultats mostren que la millor solució en quant a benefici és la inicialització buida, cosa que ens ho esperàvem ja que és la inicialització en la que es pot explorar més espai.

Tot i així, en comparació, els beneficis, són molt semblants en totes les inicialitzacions, traient el cas de la inicialització Greedy (per día de petició) que baixa molt, degut a que s'encalla fàcilment en mínims locals.

Experiment 3 - Anàlisi dels paràmetres de SA

L'objectiu d'aquest experiment és trobar els paràmetres òptims de l'algorisme Simulated Annealing amb la configuració dels experiments anteriors. Pel que podem veure en els resultats, la temperatura i la k han de ser més o menys semblants i tampoc poden ser molt grans ni molt petits, i en el nostre cas hem trobat que el valor òptim dels dos ha de ser 1000.

La λ quan més petit sembla ser que més benefici genera però el temps de computació augmenta, per tant s'ha de trobar un equilibri, i pel nostre experiment hem determinat que ha de ser 0,01. En quant al nombre d'iteracions, hem vist que el millor és 100.000.

En conclusió, hem determinat que els millors valors són $\text{num} = 100.000$, $\text{temp} = 1000$, $k = 1000$, $\lambda = 0,01$, obtenint un benefici mitjà de 86271,20 en un temps mitjà de 1,059 s.

Experiment 4 - Evolució en temps

Seguint partint de la solució inicial buida, en aquest experiment analitzem com varien el temps d'execució i el benefici quan incrementa el nombre de camions de 10 a 50 i el nombre de gasolineres de 100 a 500.

En el cas del Hill Climbing, observem que el benefici creix de manera lineal, però en canvi el temps d'execució augmenta de forma exponencial. Això era d'esperar perquè la funció generadora de successors realitza crides recursives: per a cada newBoard es tornen a calcular els seus successors.

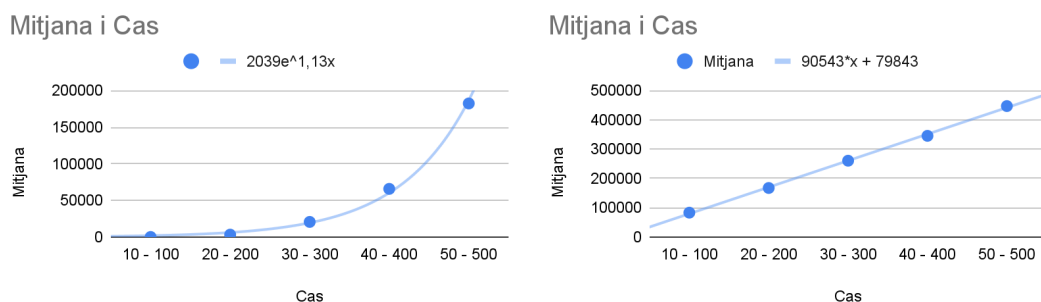


Figura 1: Gràfiques obtingudes a partir dels experiments amb HC. Esquerra: temps en ms, dreta: benefici.

En el cas del Simulated Annealing, com podem observar a la figura 2, tant el temps com els beneficis creixen linealment segons va augmentant el nombre de centres i gasolineres. Això també era previsible pel fet que l'algorisme explora solucions de manera seqüencial.

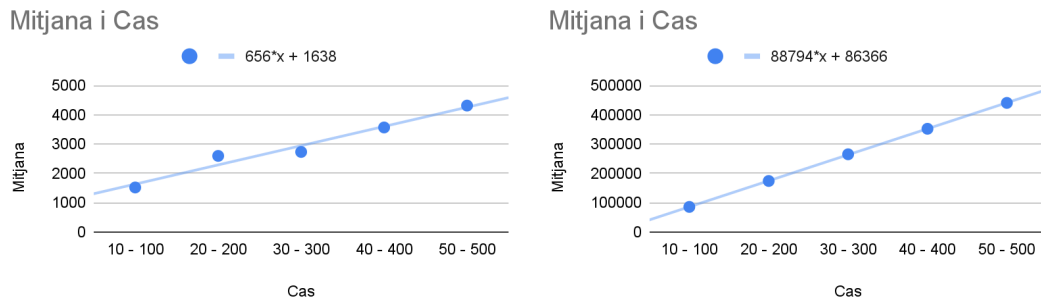


Figura 2: Gràfiques obtingudes a partir dels experiments amb SA. Esquerra: temps en ms, dreta: benefici.

Experiment 5 - Anàlisi del comportament davant d'una reducció del nombre de centres

En aquest experiment seguirem utilitzant l'algorisme de Hill Climbing i la inicialització buida per experimentar el que passaria si mantenim el nombre de camions, però reduïm el nombre de centres a la meitat, és a dir un escenari amb 5 centres amb multiplicitat 2, i per tant 10 camions, i 100 gasolineres. Aquest canvi ha fet que el temps mitjà d'execució sigui d'uns 182 ms i aconseguint una mitjana de 80871,2 de benefici.

Comparant això amb els resultats que havíem obtingut a l'experiment 2 per quan eren 10 centres (temps mitjà de 207,6 ms i benefici promig 83459), ens adonem que el fet que hi hagi menys centres de distribució, encara que mantenim el nombre de camions, afecta negativament en quant als beneficis, el temps d'execució i els quilòmetres recorreguts.

Això és degut a que, al tenir més d'un camió sortint del mateix lloc, per atendre a més peticions, com a mínim un d'ells ha de recórrer més distància per poder atendre les que estiguin més lluny i que encara no hagin estat ateses per un altre camió del mateix centre.

Experiment 6 - Anàlisi del comportament davant d'un augment en el cost per quilòmetre

L'objectiu d'aquest experiment és veure la relació entre els beneficis i el preu per km recorregut d'un camió. Seguim utilitzant Hill Climbing, comencem amb cost 2 per km com a l'escenari del primer apartat i anem doblant el cost fins a 64, on ja podem veure una tendència clara.

Amb els resultats de la figura 3 observem que els beneficis baixen d'una forma exponencial amb la pujada de preu de km recorregut d'un camió, la qual ens ho esperàvem ja que arriba un punt que no surt beneficis el fer un viatge pel cost de la distància recorreguda amb el preu del dipòsit.

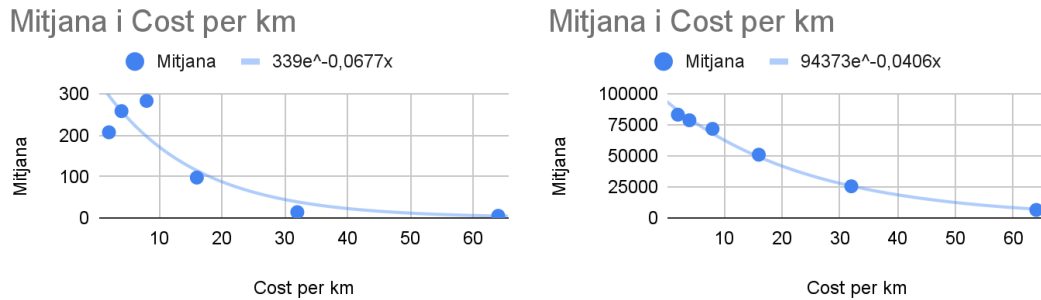


Figura 3: Gràfiques obtingudes de l'experiment 6. Esquerra: temps d'execució en ms, dreta: benefici.

Experiment 7 - Anàlisi del comportament davant d'una reducció de quilòmetres diaris

La intenció d'aquest últim experiment és veure l'efecte d'augmentar i disminuir els kilòmetres que un camió pot recórrer en un dia sobre el benefici que es pot obtenir. Mantenim el límit de 5 viatges diaris per camió i seguim amb l'escenari del primer apartat i Hill Climbing.

Dels resultats (figura 4) podem concloure quan disminueix el límit de distància diària, efectivament s'obté menys benefici. Això es deu al fet que es possible que a mesura que baixen els quilòmetres, els camions ja no puguin atendre el màxim de 10 peticions, perdent en un benefici potencial.

A mesura que puja aquest límit, també van pujant els beneficis obtinguts, però arriba a un punt en què ja no pot pujar més (com si no hi hagués aquest límit), perquè ja s'atenen a quasi totes les peticions que poden arribar a atendre. L'única opció seria per tant anar a atendre les gasolineres més llunyanes que abans no podien arribar, però això està penalitzat a l'heurística, així que no es dona el cas.

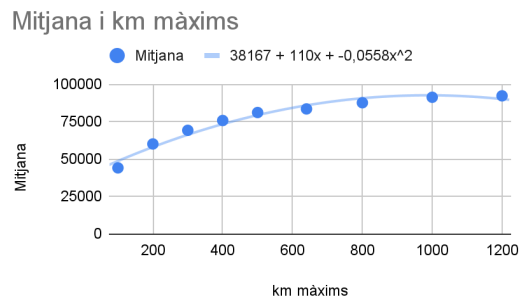


Figura 4: Gràfica obtinguda de l'experiment 7. Eix d'ordenades: benefici obtingut.

8. Conclusions

Com a conclusió, els resultats experimentals confirmen les expectatives teòriques que havíem comentat. Sobre Hill Climbing, encara que és ràpid, es veu limitat pels mínims locals, de manera que inicialitzacions massa “bones” com el Greedy 2 resulten contraproductives.

Pel que fa al Simulated Annealing, escala millor i, amb una configuració adequada dels paràmetres, obté beneficis clarament superiors als de Hill Climbing, amb una evolució del temps molt més lineal en lloc del creixement exponencial que presenta HC. Tot i que hem detectat que, a partir d'entorns amb aproximadament 500 gasolineres i 10 camions, HC pot arribar a generar beneficis lleugerament millors, aquesta opció és pràcticament inviable pels temps d'execució exagerats que implica. En resum, SA permet aconseguir beneficis molt raonables amb un cost computacional assumible. Això confirma el trade-off inevitable entre temps de càlcul i qualitat de la solució.

A més a més, hem pogut comprovar empíricament que els costos per distància, la distribució geogràfica dels centres i els límits físics dels camions afecten directament la qualitat de la solució i el temps d'execució. També ens ha deixat clar que les decisions sobre la solució inicial i els operadors disponibles són determinants per al rendiment real de la cerca.

En definitiva, la cerca local és una aproximació pragmàtica i eficient per aquest tipus de planificació, i els experiments realitzats evidencien com diferents paràmetres i restriccions modifiquen el comportament del sistema de manera previsible i mesurable.

Annex

[Annex 1] - Codi font de la pràctica (GitHub)

<https://github.com/liugetu/ia-practica1.git>

[Annex 2] - Resultats dels experiments (Google Sheet)

https://docs.google.com/spreadsheets/d/1Grxp17Fv3ozc_smunGq-9w6wUck3qXQAkuH1R5GP3Ng/edit?usp=sharing

[Annex 3] - 8.2. Competencia de trabajo en equipo: Trabajo de innovación

https://docs.google.com/document/d/1quXJp_u_u412ObRgl-mBFh7wiR4cKgpOYNZVmNP6RiI/edit?usp=sharing